

Lab 4: Number of Collisions Comparison and Recursion

Due: 11:59pm on April 13, 2025 (Sunday)

First, let's spend some time to look at number of collisions in hash table.

Download `CollisionComparison.java` file from Canvas and run it.

What is the number of collisions for hash table 1?

What is the number of collisions for hash table 2?

As you can see, we are using the same data set for those two hash tables. But, then why do we get different number of collisions for those two hash tables?

Spend some time to think about how the number of collisions is being calculated on your own. You may also find this exercise helpful for HW4.

Moving on, it is time to revisit recursion.

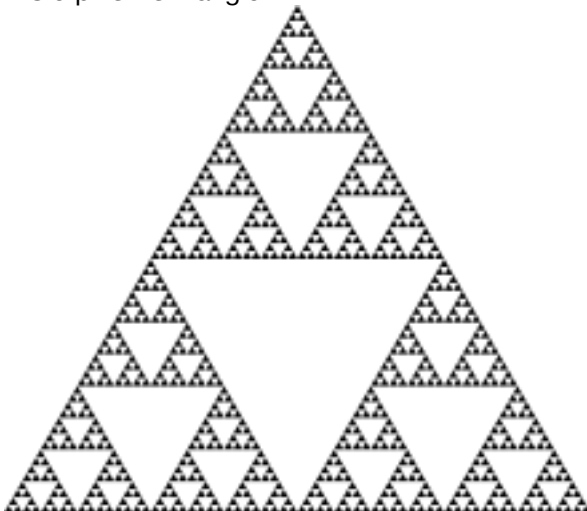
Once again, recursive definition contains a call to itself. In other words, recursive solutions are built off solutions to sub-problems, or smaller versions, of the original problem.

A fractal is an image that is self-similar.

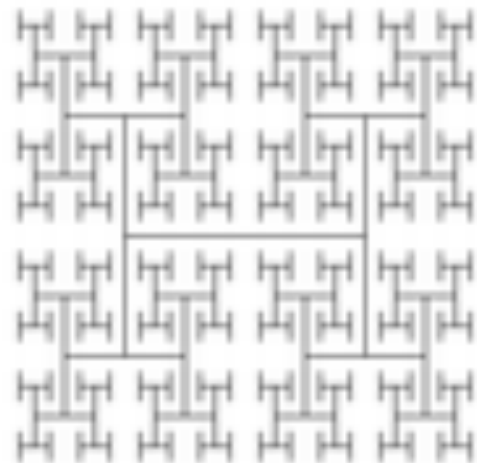
Fractals are a very typical example that can be generated using recursion.

The following is some examples of fractals.

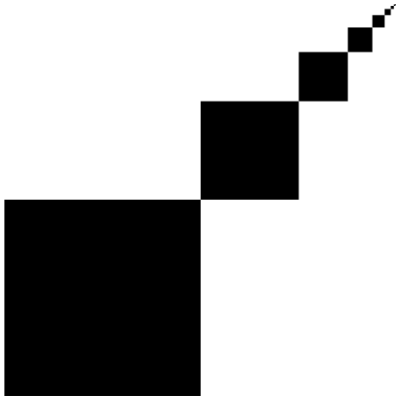
1. Sierpinski's triangle



2. H Tree



Today, let's try to generate a fractal that is shown below using recursion.



Suppose that you already have a helper method, `filledSquare(double x, double y, double radius)`, write a method to draw fractals that has `N` number of squares that are getting smaller as shown above. The height and width of squares get smaller by the half of the height and width of the previous square.

- The `filledSquare` method draws a filled square of side length $2 \times \text{radius}$, centered on (x, y) .

```
/**
 * Draw filled square of side length 2r, centered on (x, y).
 * @param x the x-coordinate of the center of the square
 * @param y the y-coordinate of the center of the square
 * @param r radius is half length of any side of the square
 * @throws RuntimeException if r is negative
 */
public static void filledSquare(double x, double y, double r){ ... }
```

- The default width and height for the canvas is 1.0.
- **Remember! Think about the base case first!**

Download the handout files (`SimpleFractal.java` and `StdDraw.java`) from Canvas and implement the recursive `fractal` method. Do not change the initial call to `fractal` in the main method.

Does your implementation draw the same fractal as in the given picture?

Submit `SimpleFractal.java` file using Autolab (<https://autolab.andrew.cmu.edu>). Do not zip the file. Autolab will only check whether your code compiles (0.5 points) and there is author comment in your source code file (0.5 points). **TAs will run your code for the remaining 2 points.**